

Análisis PRAM y Métricas de un KNN Paralelo con MPI

Ian González (100 %)

Eduardo Aragon (100 %)

Jorge Quenta (100 %)

5 de julio de 2026

Resumen

Este documento presenta el diseño y análisis teórico de una versión paralela del algoritmo K-Nearest Neighbors (KNN) usando un modelo de memoria distribuida con MPI. El algoritmo particiona la data de entrenamiento entre los procesos, replica la data de test mediante una difusión en árbol, calcula localmente los mejores K vecinos por proceso y combina los resultados mediante una reducción binaria de listas Top- K . Se derivan las expresiones teóricas de tiempo de cómputo, tiempo de comunicación, speedup, eficiencia, FLOPs y escalabilidad fuerte y débil.

1 Introducción

K-Nearest Neighbors (KNN) es un algoritmo supervisado de clasificación que asigna a un punto de prueba la etiqueta mayoritaria entre sus K vecinos más cercanos en el conjunto de entrenamiento. Dado un conjunto de entrenamiento con n_{tr} puntos, un conjunto de prueba con n_{te} puntos y una dimensión d , el costo dominante de la versión secuencial es el cálculo de distancias entre todos los puntos de prueba y todos los puntos de entrenamiento.

Esta estructura hace que KNN sea naturalmente paralelizable: las comparaciones entre un punto de prueba y distintos subconjuntos del entrenamiento pueden realizarse de manera independiente, para luego combinar los mejores candidatos parciales. En consecuencia, el grueso del trabajo puede repartirse entre procesos sin alterar la regla de decisión del clasificador.

En este proyecto, siguiendo el enfoque general planteado por el curso [1], se adopta una descomposición por datos de entrenamiento sobre memoria distribuida con MPI. Cada proceso recibe aproximadamente n_{tr}/p puntos de entrenamiento, mientras que el conjunto de prueba se replica para que cada proceso evalúe localmente sus mejores candidatos. Finalmente, los Top- K locales se fusionan mediante una reducción jerárquica y el proceso raíz emite la predicción final.

2 Método

En esta sección se presenta el diseño paralelo, su formulación PRAM, el análisis de complejidad y el vínculo con la implementación en MPI.

Símbolo	Descripción
n_{tr}	Número de puntos de entrenamiento
n_{te}	Número de puntos de prueba
d	Número de dimensiones/features
p	Número de procesos MPI
s	Identificador del proceso, con $1 \leq s \leq p$
K	Número de vecinos considerados por KNN
ℓ	Número de puntos de entrenamiento por proceso, $\ell = n_{tr}/p$
α	Latencia de envío de un mensaje
β	Tiempo de transmisión por byte
B_x	Bytes por coordenada de un punto
B_c	Bytes por candidato Top- K

Cuadro 1: Notación utilizada en el análisis.

En el análisis se asume que los datos de entrada son de tipo `float`, por lo que $B_x = 4$ bytes. Además, se representa cada candidato como un par

(distancia, índice)

donde la distancia es un `float` y el índice es un `int`. Por tanto,

$$B_c = 8 \text{ bytes.}$$

Si se utiliza `double` o una estructura con *padding*, B_c puede ser mayor.

2.1 Diseño paralelo

El algoritmo usa una descomposición por datos de entrenamiento. Cada proceso recibe un bloque del conjunto de entrenamiento y calcula, para cada punto de prueba, los mejores K vecinos dentro de su bloque local. El conjunto de prueba se difunde o replica para que todos los procesos puedan evaluar la misma consulta sobre particiones distintas del entrenamiento.

El flujo general consta de cinco etapas. Primero, se particiona el entrenamiento entre los p procesos. Segundo, se replica el conjunto de prueba mediante una difusión en árbol o una primitiva colectiva equivalente. Tercero, cada proceso calcula su Top- K local para cada punto de prueba. Cuarto, las listas Top- K locales se fusionan mediante una reducción binaria. Quinto, el proceso raíz realiza la votación mayoritaria sobre el Top- K global y genera las predicciones finales.

La operación de combinación entre dos listas Top- K se define como:

$$\text{MergeTopK}(A, B, K) = \text{mejores } K \text{ candidatos de } A \cup B.$$

Esta operación permite una reducción porque los mejores K vecinos globales deben estar dentro de la unión de los mejores K vecinos locales de cada proceso.

2.2 PRAM del algoritmo

El siguiente pseudocódigo resume el diseño paralelo propuesto en términos de operaciones por proceso y explicita la etapa de reducción binaria de listas Top- K .

Algorithm 1: KNN paralelo por proceso con reducción binaria de Top- K

Input: $X_{tr}[1, \dots, n_{tr}][1, \dots, d]$, $Y_{tr}[1, \dots, n_{tr}]$, $X_{te}[1, \dots, n_{te}][1, \dots, d]$, $p = 2^q$ procesos, proceso s con $1 \leq s \leq p$, número de vecinos K

Output: Predicciones $Pred[1, \dots, n_{te}]$ almacenadas en el proceso $s = 1$

```
1  $\ell \leftarrow n_{tr}/p$ ;
   // 1. Distribución del conjunto de entrenamiento
2 if  $s = 1$  then
3   for  $r \leftarrow 1$  to  $p$  do
4     enviar  $X_{tr}[\ell(r-1)+1, \dots, \ell r][1, \dots, d]$  al proceso  $r$ ;
5     enviar  $Y_{tr}[\ell(r-1)+1, \dots, \ell r]$  al proceso  $r$ ;
6 recibir LocalX[1, ...,  $\ell$ ][1, ...,  $d$ ];
7 recibir LocalY[1, ...,  $\ell$ ];
   // 2. Difusión en árbol del conjunto de prueba
8 for  $h \leftarrow 1$  to  $\log_2(p)$  do
9   half  $\leftarrow 2^{h-1}$ ;
10  if  $s \leq \text{half}$  then
11    dest  $\leftarrow s + \text{half}$ ;
12    if dest  $\leq p$  then
13      enviar  $X_{te}[1, \dots, n_{te}][1, \dots, d]$  a dest;
14  else if half  $< s \leq 2^h$  then
15    src  $\leftarrow s - \text{half}$ ;
16    recibir  $X_{te}[1, \dots, n_{te}][1, \dots, d]$  desde src;
   // 3. Cálculo local de Top- $K$ 
17 for  $t \leftarrow 1$  to  $n_{te}$  do
18   TopKs[ $t$ ]  $\leftarrow \emptyset$ ;
19   for  $i \leftarrow 1$  to  $\ell$  do
20     dist  $\leftarrow \text{distance}(X_{te}[t], \text{LocalX}[i])$ ;
21     idx  $\leftarrow \ell(s-1) + i$ ;
22     candidate  $\leftarrow (\text{dist}, \text{idx})$ ;
23     TopKs[ $t$ ]  $\leftarrow \text{InsertAndKeepBestK}(\text{TopK}_s[t], \text{candidate}, K)$ ;
   // 4. Reducción binaria de Top- $K$ 
24 for  $h \leftarrow 1$  to  $\log_2(p)$  do
25   block_size  $\leftarrow 2^h$ ;
26   half  $\leftarrow 2^{h-1}$ ;
27   if  $(s-1) \bmod \text{block\_size} = \text{half}$  then
28     parent  $\leftarrow s - \text{half}$ ;
29     enviar TopKs[1, ...,  $n_{te}$ ] a parent;
30     break;
31   else if  $(s-1) \bmod \text{block\_size} = 0$  then
32     child  $\leftarrow s + \text{half}$ ;
33     if child  $\leq p$  then
34       recibir TopKchild[1, ...,  $n_{te}$ ] desde child;
35       for  $t \leftarrow 1$  to  $n_{te}$  do
36         TopKs[ $t$ ]  $\leftarrow \text{MergeTopK}(\text{TopK}_s[t], \text{TopK}_{child}[t], K)$ ;
   // 5. Votación mayoritaria en el proceso raíz
37 if  $s = 1$  then
38   for  $t \leftarrow 1$  to  $n_{te}$  do
39     Pred[ $t$ ]  $\leftarrow \text{MajorityVote}(\text{TopK}_s[t], Y_{tr})$ 
```

En la fase de reducción, cada proceso activo representa el acumulador del subárbol izquierdo. Por ello, en cada nivel h , solo se envía la información del subárbol derecho hacia el proceso padre. El proceso padre fusiona su TopK_s actual con el TopK recibido y conserva los mejores K candidatos para cada punto de prueba.

La condición de envío es:

$$(s - 1) \bmod 2^h = 2^{h-1},$$

mientras que la condición de recepción es:

$$(s - 1) \bmod 2^h = 0.$$

Para $p = 8$, los envíos son:

$$h = 1 : \quad 2 \rightarrow 1, 4 \rightarrow 3, 6 \rightarrow 5, 8 \rightarrow 7,$$

$$h = 2 : \quad 3 \rightarrow 1, 7 \rightarrow 5,$$

$$h = 3 : \quad 5 \rightarrow 1.$$

Al finalizar, el proceso $s = 1$ contiene el Top-K global para cada punto de prueba.

2.3 Complejidad de cómputo

2.3.1 Cálculo local de distancias

Cada proceso compara sus $\ell = n_{tr}/p$ puntos de entrenamiento contra los n_{te} puntos de prueba. Cada distancia euclidiana en dimensión d cuesta:

$$O(d).$$

Se asume que la lista Top-K de cada punto de prueba se mantiene con una cola de prioridad de tamaño máximo K . Por tanto, insertar o actualizar un candidato cuesta:

$$O(\log K).$$

Así, el costo local principal por proceso es:

$$T_{\text{local}} = O\left(\frac{n_{tr}}{p} n_{te} (d + \log K)\right).$$

2.3.2 Cómputo durante la reducción

En cada nivel de la reducción, un proceso receptor fusiona dos listas Top-K para cada punto de prueba. Fusionar dos listas de tamaño K cuesta:

$$O(K).$$

Como hay n_{te} puntos de prueba, el costo por nivel es:

$$O(n_{te} K).$$

La reducción tiene profundidad $\log_2 p$, por lo que:

$$T_{\text{reduce,comp}} = O(n_{te} K \log p).$$

2.3.3 Votación mayoritaria

En el proceso raíz, la votación mayoritaria se realiza para cada punto de prueba usando sus K vecinos globales:

$$T_{\text{vote}} = O(n_{te} K).$$

2.3.4 Tiempo total de cómputo

Por tanto, el tiempo de cómputo paralelo es:

$$T_{\text{comp}} = O\left(\frac{n_{tr}n_{te}}{p}(d + \log K) + n_{te}K \log p + n_{te}K\right).$$

Como el término de votación suele estar dominado por el cálculo de distancias, puede simplificarse como:

$$T_{\text{comp}} = O\left(\frac{n_{tr}n_{te}}{p}(d + \log K) + n_{te}K \log p\right).$$

2.4 Complejidad de comunicación

El costo de comunicación se modela como:

$$T_{\text{msg}} = \alpha + x\beta,$$

donde α es la latencia, β es el costo por byte transmitido y x es el número de bytes enviados.

2.4.1 Distribución del conjunto de entrenamiento

El proceso raíz distribuye aproximadamente $\ell = n_{tr}/p$ puntos a cada proceso. Cada punto tiene d coordenadas y cada coordenada ocupa $B_x = 4$ bytes. La cantidad total de bytes enviados es:

$$4n_{tr}d.$$

Además, hay p mensajes. Por tanto:

$$T_{\text{scatter}} = O(p\alpha + 4\beta n_{tr}d).$$

Si se incluyen las etiquetas Y_{tr} , debe añadirse:

$$O(4\beta n_{tr}),$$

asumiendo etiquetas enteras de 4 bytes. Como usualmente $d \gg 1$, este término es menor.

2.4.2 Difusión en árbol del conjunto de prueba

El conjunto de prueba contiene $n_{te}d$ valores. Como cada valor ocupa 4 bytes:

$$x = 4n_{te}d.$$

Usando difusión en árbol, la profundidad es $\log p$, por lo que:

$$T_{\text{bcast}} = O((\alpha + 4\beta n_{te}d) \log p).$$

2.4.3 Reducción binaria de listas Top- K

Cada proceso envía, para cada punto de prueba, una lista de K candidatos. Cada candidato contiene distancia e índice, por lo que ocupa $B_c = 8$ bytes. El tamaño del mensaje de reducción es:

$$x = 8n_{te}K.$$

Como la reducción tiene profundidad $\log p$:

$$T_{\text{reduce,comm}} = O((\alpha + 8\beta n_{te}K) \log p).$$

2.4.4 Tiempo total de comunicación

Sumando los términos:

$$T_{\text{comm}} = O(p\alpha + 4\beta n_{tr}d + (\alpha + 4\beta n_{te}d) \log p + (\alpha + 8\beta n_{te}K) \log p).$$

2.5 Métricas teóricas

2.5.1 Tiempo paralelo total

El tiempo paralelo total es:

$$T_p = T_{\text{comp}} + T_{\text{comm}}.$$

Sustituyendo:

$$T_p = O\left(\frac{n_{tr}n_{te}}{p}(d + \log K) + n_{te}K \log p + p\alpha + 4\beta n_{tr}d + (\alpha + 4\beta n_{te}d) \log p + (\alpha + 8\beta n_{te}K) \log p\right).$$

2.5.2 Tiempo secuencial

En la versión secuencial, se comparan todos los puntos de prueba contra todos los puntos de entrenamiento. Con una cola de prioridad de tamaño K , se obtiene:

$$T_1 = O(n_{tr}n_{te}(d + \log K) + n_{te}K).$$

Como $n_{tr}d$ suele ser mucho mayor que K , normalmente se simplifica como:

$$T_1 = O(n_{tr}n_{te}(d + \log K)).$$

2.5.3 Speedup teórico y eficiencia

El speedup se define como:

$$S_p = \frac{T_1}{T_p}.$$

Entonces:

$$S_p = \frac{O(n_{tr}n_{te}(d + \log K))}{O\left(\frac{n_{tr}n_{te}}{p}(d + \log K) + n_{te}K \log p + T_{\text{comm}}\right)}.$$

Si se ignora comunicación y reducción, el speedup ideal sería:

$$S_p^{\text{ideal}} = p.$$

Sin embargo, en la práctica:

$$S_p < p,$$

debido a comunicación, reducción y desbalance de carga.

La eficiencia se define como:

$$E_p = \frac{S_p}{p}.$$

Sustituyendo:

$$E_p = \frac{T_1}{pT_p}.$$

La eficiencia mide qué fracción del rendimiento ideal se aprovecha al usar p procesos. Cuando $E_p \approx 1$, el paralelismo es casi ideal. Cuando E_p disminuye, significa que la comunicación, sincronización o el desbalance de carga están dominando.

2.5.4 FLOPs

Para dos puntos $x, y \in \mathbb{R}^d$, se calcula la distancia euclidiana al cuadrado:

$$\|x - y\|^2 = \sum_{j=1}^d (x_j - y_j)^2.$$

Para cada dimensión se realizan aproximadamente:

- 1 resta: $x_j - y_j$,
- 1 multiplicación: $(x_j - y_j)^2$,
- 1 suma acumulada.

Por tanto, se cuentan aproximadamente:

$$3d$$

FLOPs por distancia.

No se calcula la raíz cuadrada, porque para KNN basta comparar distancias al cuadrado; la raíz cuadrada preserva el orden de las distancias y añade costo innecesario.

La región paralelizable principal calcula:

$$n_{tr}n_{te}$$

distancias. Por tanto:

$$\boxed{\text{FLOPs}_{total} = 3n_{tr}n_{te}d}.$$

Por proceso:

$$\boxed{\text{FLOPs}_{proc} = \frac{3n_{tr}n_{te}d}{p}}.$$

Si T_{comp}^{exp} es el tiempo experimental de cómputo medido, entonces:

$$\boxed{\text{FLOPs/s} = \frac{3n_{tr}n_{te}d}{T_{comp}^{exp}}}.$$

Si se desea calcular GFLOPs/s:

$$\boxed{\text{GFLOPs/s} = \frac{3n_{tr}n_{te}d}{10^9 T_{comp}^{exp}}}.$$

2.5.5 Escalabilidad teórica

La escalabilidad fuerte evalúa cómo cambia el tiempo al aumentar p manteniendo fijo el tamaño del problema:

$$n_{tr}, n_{te}, d, K = \text{constantes}.$$

En este caso:

$$T_{comp} = O\left(\frac{n_{tr}n_{te}}{p}(d + \log K)\right)$$

disminuye al aumentar p .

Sin embargo, los términos de comunicación:

$$p\alpha,$$

$$(\alpha + 4\beta n_{te}d) \log p,$$

$$(\alpha + 8\beta n_{te}K) \log p$$

no disminuyen con p . Por tanto, para valores pequeños de p , el algoritmo debería mostrar mejoras cercanas a lineales. Para valores grandes de p , el costo de comunicación y reducción empieza a dominar.

El comportamiento esperado es:

$$S_p \approx p \quad \text{para } p \text{ pequeño,}$$

pero:

$$E_p = \frac{S_p}{p}$$

decrece cuando p crece demasiado.

Una aproximación útil consiste en escribir:

$$T_p \approx \frac{A}{p} + C \log p + Dp,$$

donde:

$$A = n_{tr}n_{te}(d + \log K),$$

$$C \approx n_{te}K + \alpha + 4\beta n_{te}d + 8\beta n_{te}K,$$

y D representa el costo asociado a términos lineales como $p\alpha$.

El número óptimo de procesos se estima experimentalmente como:

$$\boxed{p^* = \arg \min_p T_p^{exp}}.$$

También puede definirse como el mayor p antes de que la eficiencia caiga bajo un umbral, por ejemplo:

$$E_p < 0.5.$$

La escalabilidad débil evalúa qué ocurre cuando aumentan simultáneamente el tamaño del problema y el número de procesos.

Para este diseño, como se particiona con respecto a n_{tr} , una forma natural de escalabilidad débil es mantener constante la cantidad de entrenamiento por proceso:

$$\frac{n_{tr}}{p} = \ell = \text{constante}.$$

Si además se mantiene fijo n_{te} , d y K , entonces:

$$T_{\text{comp}} = O(\ell n_{te}(d + \log K)),$$

que es constante con respecto a p .

Pero la comunicación aún crece:

$$T_{\text{broadcast}} = O((\alpha + 4\beta n_{te}d) \log p),$$

$$T_{\text{reduce}} = O((\alpha + 8\beta n_{te}K) \log p).$$

Por tanto, bajo esta forma de *weak scaling*:

$$T_p = O(1 + \log p).$$

Esto implica que el algoritmo puede escalar razonablemente, pero no de forma perfecta, debido a la comunicación global de X_{te} y la reducción de Top- K .

Si se escala todo el dataset manteniendo fija la proporción entre entrenamiento y prueba:

$$n_{tr} \propto p, \quad n_{te} \propto p,$$

entonces el trabajo total de KNN crece como:

$$O(n_{tr}n_{te}d) = O(p^2d).$$

Al dividir solo n_{tr} entre p procesos:

$$T_{\text{comp}} = O\left(\frac{n_{tr}n_{te}d}{p}\right) = O(pd).$$

Por tanto, si tanto n_{tr} como n_{te} crecen proporcionalmente a p , el tiempo por proceso no se mantiene constante, sino que crece linealmente con p . Esto no representa *weak scaling* ideal, pero sí representa el comportamiento esperado cuando se aumenta el tamaño completo del dataset.

La notación Big- O describe crecimiento asintótico, pero no da tiempos en segundos. Para comparar la teoría con los experimentos, se normaliza la expresión teórica usando una constante de proporcionalidad. Se propone ajustar:

$$T_{\text{theory}}(p) = C_1 \frac{n_{tr}n_{te}}{p} (d + \log K) + C_2 n_{te} K \log p + C_3 T_{\text{comm}}(p),$$

donde C_1 , C_2 y C_3 se estiman a partir de los resultados experimentales.

Una forma simple de normalizar es usar el tiempo experimental para $p = 1$:

$$C_1 = \frac{T_{\text{exp}}(1)}{n_{tr}n_{te}(d + \log K)}.$$

Luego se grafica:

$$T_{\text{theory}}(p) \quad \text{vs.} \quad T_{\text{exp}}(p).$$

2.6 Análisis de isoeficiencia (condición de escalabilidad)

En este curso la **isoeficiencia** se evalúa como una *condición de escalabilidad débil* (weak scaling) basada en la Ley de Gustafson. No se busca una función de isoeficiencia aislada, sino la relación que dicta cómo debe crecer el tamaño del problema n en función del número de procesos p para que la eficiencia E se mantenga constante ($E = O(1)$). Formalizamos aquí la condición que ya se anticipó de manera cualitativa en las secciones de escalabilidad fuerte y débil, y comparamos el paradigma usado (MPI) con el de memoria compartida (OMP/PRAM).

2.6.1 Función de overhead y forma canónica de la eficiencia

Para obtener una única condición de escalabilidad tomamos un tamaño de problema n con $n_{tr} = n_{te} = n$ (el dataset completo escala). De las complejidades ya derivadas, el trabajo total (tiempo secuencial dominante, omitiendo $\log K$) es

$$W \equiv T_1 = O(n^2d),$$

y el cómputo paralelo reparte ese trabajo idealmente, $T_{\text{comp}} = O(n^2d/p)$.

Definimos la **función de overhead total** T_o como el trabajo que no realiza el algoritmo secuencial:

$$T_o(p) = p T_p - W = p T_{\text{comp}} + p T_{\text{comm}} - W.$$

Como $p T_{\text{comp}} = W$, todo el overhead proviene de la comunicación y la sincronización:

$$\boxed{T_o(p) = p T_{\text{comm}}.}$$

Sustituyendo en la eficiencia se obtiene la forma canónica del curso:

$$E = \frac{S_p}{p} = \frac{T_1}{p T_p} = \frac{W}{W + T_o} = \frac{1}{1 + \frac{T_o}{W}}.$$

Para que $E = O(1)$ sea constante, el término de overhead T_o/W debe permanecer constante: esa igualdad, despejada, define la condición de isoeficiencia.

2.6.2 Isoeficiencia en MPI (memoria distribuida)

En MPI la comunicación es explícita y es el cuello de botella. Sustituyendo $n_{tr} = n_{te} = n$ en T_{comm} y conservando los términos dominantes:

$$T_o = p T_{\text{comm}} \approx \underbrace{p^2 \alpha}_{(1) \text{ latencia}} + \underbrace{p \beta n d}_{(2) \text{ scatter}} + \underbrace{p \beta n d \log p}_{(3) \text{ bcast/reducción}}.$$

El término (3) proviene de difundir el conjunto de prueba ($O(nd)$ bytes) y de reducir las listas Top- K a lo largo de una jerarquía de profundidad $\log p$; domina sobre (2). Imponemos $T_o/W = \text{const}$ con $W = n^2 d$:

$$\begin{aligned} (1) \quad \frac{p^2 \alpha}{n^2 d} = \text{const} &\Rightarrow n \propto p, & (2) \quad \frac{p \beta n d}{n^2 d} = \frac{p \beta}{n} = \text{const} &\Rightarrow n \propto p, \\ (3) \quad \frac{p \beta n d \log p}{n^2 d} = \frac{p \beta \log p}{n} = \text{const} &\Rightarrow n \propto p \log p. \end{aligned}$$

La condición más restrictiva es la del término (3). Por tanto:

Isoeficiencia MPI: $n \propto p \log p$, $W_{iso} = \Theta(p^2 \log^2 p)$.

Es decir, el dataset debe crecer ligeramente más rápido que p (factor $\log p$) para amortizar la difusión del test y la reducción global de candidatos sobre la red. Si la reducción se hiciera con un **gather** plano al proceso raíz en vez de la reducción en árbol, el término dominante sería $O(p)$ en lugar de $O(\log p)$, saturando el enlace del root y empeorando notablemente la isoeficiencia.

2.6.3 Isoeficiencia en OMP / PRAM (memoria compartida)

En OpenMP los hilos comparten el mismo espacio de direcciones (modelo UMA), lo que cambia el overhead de forma fundamental:

- **No hay difusión de datos:** el conjunto de prueba es visible por todos los hilos, por lo que *desaparecen* los términos $p\alpha$ y $\beta n d$ de la comunicación explícita ($T_{\text{comm}} \approx 0$).
- El único overhead proviene de la **sincronización de hilos** (barreras) y de la reducción de listas Top- K de profundidad $O(\log p)$, realizada mediante accesos directos a memoria.

Modelando ese overhead (con K constante):

$$T_o^{\text{omp}} \approx p \cdot O(n_{te} K \log p) + O(p) = O(p n \log p) + O(p).$$

Imponiendo $T_o^{\text{omp}}/W = \text{const}$ con $W = n^2 d$, y observando que ya no aparece el factor de transporte de datos βd :

$$\frac{p n \log p}{n^2 d} = \frac{p \log p}{n d} = \text{const} \Rightarrow n \propto \frac{p \log p}{d}.$$

Isoeficiencia OMP: $n \propto \frac{p \log p}{d}$ (a lo sumo $n \propto p$), $W_{iso} = \Theta(p^2)$ sin la constante de red β .

OMP tiene una isoeficiencia asintóticamente mejor (no paga latencia ni ancho de banda de red), **pero** p está estrictamente acotado al número de núcleos físicos del nodo y todo el dataset debe caber en su RAM.

2.6.4 Comparación y elección del paradigma

Criterio	MPI (memoria distribuida)	OMP (memoria compartida)
Comunicación T_{comm}	Explícita: α, β (red)	≈ 0 (acceso directo)
Overhead dominante	Difusión/reducción $\beta n d \log p$	Sincronización (barreras)
Condición de isoeficiencia	$n \propto p \log p$	$n \propto p \log p / d$ (más suave)
Trabajo W_{iso}	$\Theta(p^2 \log^2 p)$	$\Theta(p^2)$ sin β
Límite de p	Horizontal (varios nodos)	Núcleos de un solo nodo
Límite de n	Suma de RAM de los nodos	RAM de un solo nodo

Cuadro 2: Comparación de la escalabilidad de ambos paradigmas para el KNN.

Aunque OMP escala mejor asintóticamente, su paralelismo está limitado a los núcleos y la memoria de un solo nodo. En este proyecto se eligió **MPI por un tema de memoria**: al distribuir el conjunto de entrenamiento, el mismo código puede superar el límite de RAM de un nodo y escalar n y p horizontalmente sin reescritura. Los experimentos evaluaron hasta $p = 32$ procesos, y la elección del paradigma de memoria distribuida es la que habilita crecer más allá de ese punto cuando el dataset no cabe en un único nodo.

2.7 Implementación

La implementación sigue directamente el diseño PRAM anterior y se desarrolló en Python usando `mpi4py` [2]. Para la carga y validación inicial de datos se aprovecha la infraestructura de `scikit-learn` [3]. El repositorio del proyecto se encuentra en <https://github.com/mukanjy0/par-knn> [4].

Las versiones activas del código se organizaron con dos objetivos distintos. La **v2** implementa una estrategia de comunicación más explícita o manual: distribución del entrenamiento mediante envíos directos, difusión del conjunto de prueba con un árbol explícito y reducción binaria manual de listas Top- K . La **v3**, en cambio, conserva la misma idea algorítmica pero reemplaza las etapas que corresponden por primitivas colectivas de MPI, en particular `Scatterv`, `Bcast` y `Reduce` con una operación personalizada de fusión de candidatos.

Fase del diseño	Implementación MPI representativa
Distribución de entrenamiento	Send/Recv en v2; Scatterv en v3
Difusión de test	árbol explícito en v2; Bcast en v3
Cálculo local	Código local NumPy/Python
Reducción Top- K	árbol binario manual en v2; Reduce personalizada en v3
Votación final	Proceso root

Cuadro 3: Correspondencia entre el diseño teórico y las dos implementaciones MPI analizadas.

Como apoyo metodológico, se utilizaron asistentes de IA como Claude y Codex [5, 6] para acelerar tareas de implementación, depuración, automatización experimental y revisión del documento. En particular, se emplearon para traducir el diseño PRAM a código MPI/`mpi4py`, depurar y comparar las versiones v2/v3, preparar scripts de ejecución y trabajos SLURM, organizar resultados desde CSV en tablas y figuras, y revisar el formato LaTeX. El diseño algorítmico, la selección de métricas y la interpretación final de resultados fueron responsabilidad del equipo; estas herramientas se usaron como apoyo auxiliar, no como fuentes teóricas principales ni como sustitutos del análisis.

3 Resultados y análisis

Esta sección resume los resultados medidos para las implementaciones v2 y v3. Las tablas se construyen a partir de los archivos `summary.csv` de `report/results`; las filas crudas de `raw_results.csv` se usan solo para contextualizar repeticiones y fases de comunicación.

3.1 Entorno experimental

Los experimentos usan el dataset *digits* de *scikit-learn*, con $d = 64$ features por imagen y $K = 3$. Se evaluaron dos implementaciones MPI: v2, basada en comunicación más explícita, y v3, basada en primitivas colectivas de MPI cuando corresponde. Las etiquetas del reporte se toman de la carpeta de resultados; la columna `command` conserva nombres históricos de versiones y no se usa para identificar implementaciones.

El dataset original *digits* no contiene suficientes muestras para los tamaños usados en escalabilidad. Por ello, el cargador divide primero el dataset original en train/test de forma estratificada y luego expande cada split por separado. Cuando el tamaño objetivo supera el tamaño del split original, se remuestran ejemplos del mismo split y se les añade ruido gaussiano controlado por la semilla de aumento. Así, valores como $n = 15000, \dots, 105000$ representan tamaños experimentales para medir escalabilidad, no nuevos datos independientes del dataset original. La separación train/test se realiza antes del aumento para evitar fuga de datos entre ambos splits.

En strong scaling se fijó $n_{test} = 5000$ y se evaluaron cuatro tamaños de entrenamiento: 10000, 25000, 50000 y 100000, equivalentes a $n = 15000, 30000, 55000$ y 105000. En weak scaling se mantuvo $n_{train}/p = 3125$ y $n_{test} = 5000$, con $p = 1, 2, 4, 8, 16, 32$. En la prueba de accuracy se usaron $n = 10000, 50000$ y 100000 con partición 80/20, y $p = 1, 8, 32$.

Los experimentos de strong y weak scaling tienen 3 repeticiones por configuración; los de accuracy tienen 1 ejecución. Por ello, las tablas de rendimiento reportan principalmente medianas de tiempo y FLOPs/s, mientras que la accuracy se reporta como el promedio disponible en `summary.csv`.

3.2 Validación de correctitud

La Tabla 4 muestra que la accuracy se mantiene constante al variar p para cada tamaño evaluado. Además, v2 y v3 reportan exactamente la misma accuracy en todas las configuraciones. Esto indica que la paralelización preserva el comportamiento del KNN: cambiar el número de procesos modifica el tiempo de ejecución, pero no la clasificación resultante.

n	p	Accuracy v2	Accuracy v3	Predicciones
10000	1	0.98900	0.98900	iguales
10000	8	0.98900	0.98900	iguales
10000	32	0.98900	0.98900	iguales
50000	1	0.98550	0.98550	iguales
50000	8	0.98550	0.98550	iguales
50000	32	0.98550	0.98550	iguales
100000	1	0.98675	0.98675	iguales
100000	8	0.98675	0.98675	iguales
100000	32	0.98675	0.98675	iguales

Cuadro 4: Validación de correctitud: la accuracy no cambia con p y coincide entre v2 y v3. Los archivos `.npy` de predicciones para $p = 1, 8, 32$ son idénticos byte a byte dentro de cada versión.

3.3 Comparación de implementaciones: v2 vs v3

La comparación se realiza con el escenario strong scaling de mayor tamaño, $n = 105000$. La Tabla 5 muestra que v2 obtiene menor tiempo total para todos los valores de p . Por ejemplo, con $p = 16$, v2 alcanza 1.944 s frente a 3.097 s de v3; con $p = 32$, v2 obtiene 2.022 s frente a 2.461 s de v3. En cambio, v3 presenta speedups relativos mayores porque su baseline de $p = 1$ es considerablemente más lento.

p	T_{v2} (s)	T_{v3} (s)	S_{v2}	S_{v3}	E_{v2}	E_{v3}	Accuracy
1	11.119	54.918	1.00	1.00	1.00	1.00	0.9880
2	5.835	26.361	1.91	2.08	0.95	1.04	0.9880
4	3.028	11.917	3.67	4.61	0.92	1.15	0.9880
8	2.088	5.701	5.33	9.63	0.67	1.20	0.9880
16	1.944	3.097	5.72	17.73	0.36	1.11	0.9880
32	2.022	2.461	5.50	22.32	0.17	0.70	0.9880

Cuadro 5: Comparación v2 vs v3 para strong scaling con $n = 105000$. v2 domina en tiempo absoluto; v3 obtiene mayor speedup relativo por partir de un tiempo base más alto.

Las eficiencias mayores que 1 en v3 no se interpretan como superioridad práctica ni como escalabilidad física ideal. En este reporte, la eficiencia se calcula respecto al tiempo medido con $p = 1$ de la misma versión. Como v3 tiene un baseline con $p = 1$ mucho más lento que v2 (54.918 s frente a 11.119 s para $n = 105000$), el cociente puede producir speedups y eficiencias relativas mayores que 1 en algunos puntos. Por esa razón, la comparación entre implementaciones se decide por tiempo absoluto, estabilidad de accuracy y tendencia general, no solo por speedup normalizado.

Para el resto del análisis se selecciona **v2**. La decisión se basa en el tiempo total, que es menor en todos los casos de strong scaling evaluados, y en que la accuracy se mantiene estable. Aunque v3 tiene mejores speedups relativos, ese indicador queda condicionado por un tiempo secuencial/paralelo con $p = 1$ más alto, por lo que no representa una ventaja práctica en tiempo absoluto para estos datos.

3.4 Strong scaling

El análisis de strong scaling usa v2 y el mayor tamaño evaluado, $n = 105000$, con $n_{train} = 100000$ y $n_{test} = 5000$. La Tabla 6 muestra que el tiempo total baja de 11.119 s con $p = 1$ a 1.944 s con $p = 16$. Al pasar a $p = 32$, el tiempo sube ligeramente a 2.022 s, señal de saturación.

También se evaluaron otros tamaños de problema ($n = 15000$, 30000 y 55000). El punto óptimo no es fijo: depende de la cantidad de trabajo disponible para amortiguar comunicación y overhead. Por ello, la Tabla 12 se usa más adelante como soporte agregado para comparar el mejor p por tamaño.

n	p	T_{total} (s)	T_{comp} (s)	T_{comm} (s)	Speedup	Eficiencia
105000	1	11.119	11.086	0.033	1.00	1.00
105000	2	5.835	5.813	0.023	1.91	0.95
105000	4	3.028	3.001	0.027	3.67	0.92
105000	8	2.088	2.044	0.030	5.33	0.67
105000	16	1.944	1.910	0.035	5.72	0.36
105000	32	2.022	1.950	0.072	5.50	0.17

Cuadro 6: Strong scaling de v2 para $n = 105000$. El tiempo de cómputo cae al aumentar p , pero el speedup se satura entre $p = 16$ y $p = 32$.

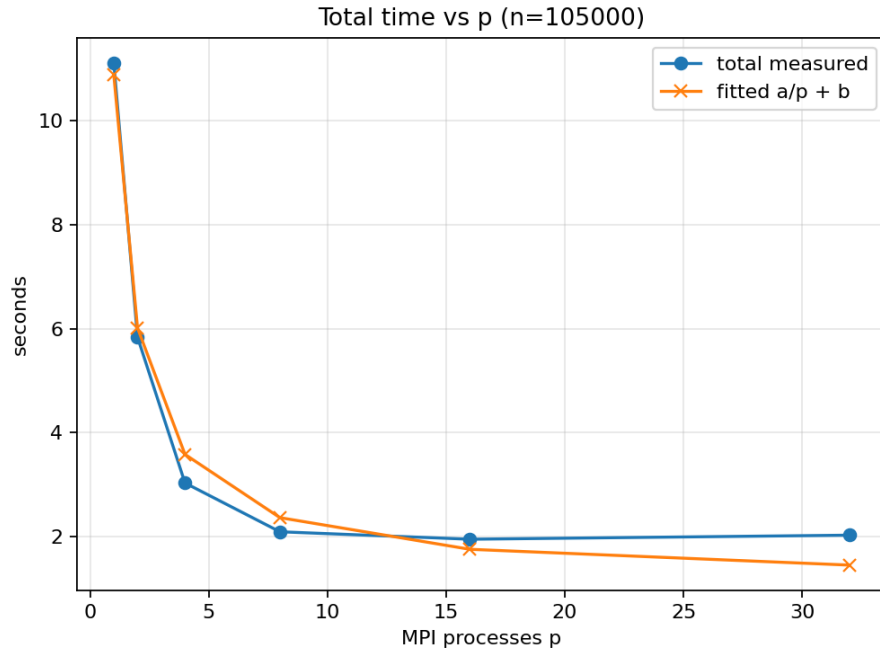


Figura 1: Tiempo total de v2 para strong scaling con $n = 105000$: la reducción es clara hasta $p = 16$, pero $p = 32$ no mejora el tiempo mínimo.

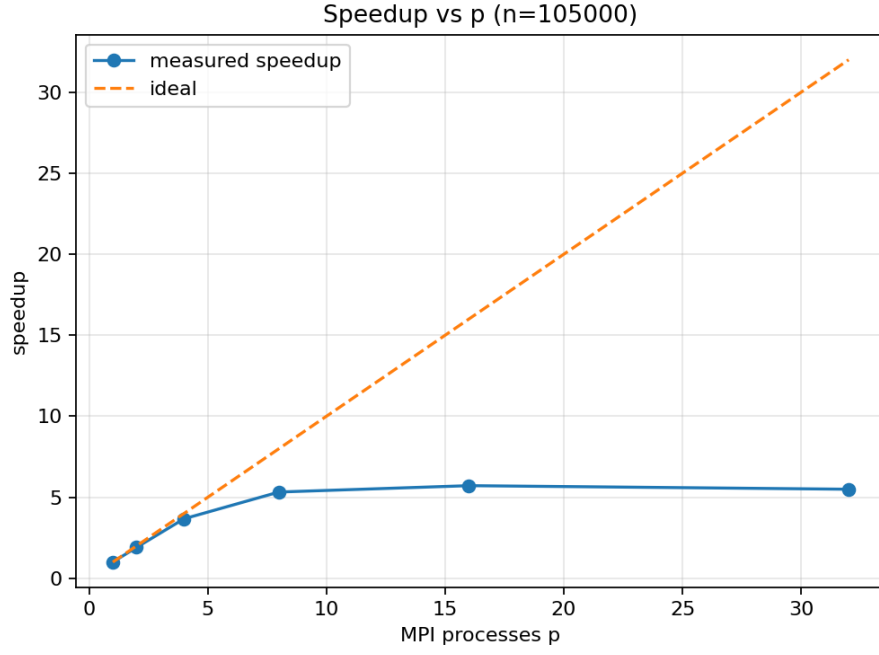


Figura 2: Speedup de v2 para $n = 105000$: el crecimiento deja de ser cercano al ideal cuando el costo no paralelizable y la comunicación pesan más.

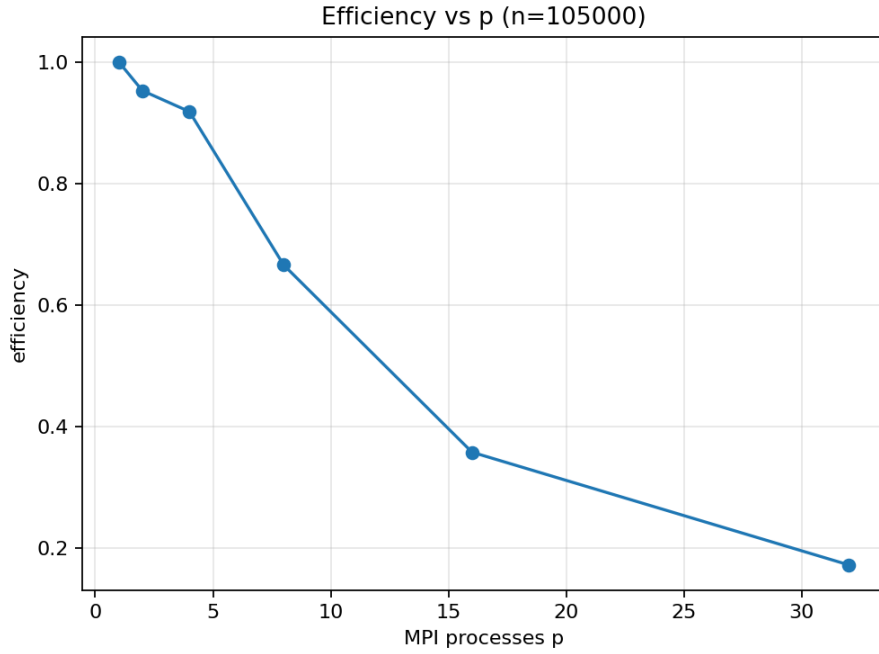


Figura 3: Eficiencia de v2 para $n = 105000$: la eficiencia baja de 0.67 en $p = 8$ a 0.17 en $p = 32$, mostrando pérdida de aprovechamiento de procesos.

La tendencia coincide con el modelo teórico. Si se aproxima el tiempo como $T_p \approx A/p + C \log p + Dp$, el primer término explica la mejora inicial: al aumentar p , cada proceso calcula distancias contra una fracción menor del entrenamiento. Los términos $C \log p$ y Dp representan comunicación, reducción y overhead que no decrecen con p . En las curvas medidas, esto se manifiesta en una caída fuerte del tiempo hasta $p = 8$ – 16 y una saturación posterior, exactamente

el comportamiento esperado cuando el término A/p deja de dominar.

3.5 Weak scaling

En los datos de weak scaling de v2 se verifica que $n_{train}/p = 3125$ para todos los valores de p , mientras que $n_{test} = 5000$ se mantiene constante. Por tanto, el experimento corresponde al weak scaling natural de este diseño: la carga de entrenamiento por proceso permanece fija.

p	n	n_{train}	n_{test}	n_{train}/p	T_{total} (s)	GFLOPs/s
1	8125	3125	5000	3125	0.243	12.41
2	11250	6250	5000	3125	0.387	15.78
4	17500	12500	5000	3125	0.406	30.23
8	30000	25000	5000	3125	0.529	46.76
16	55000	50000	5000	3125	0.917	53.93
32	105000	100000	5000	3125	2.019	49.92

Cuadro 7: Weak scaling de v2. La carga n_{train}/p se mantiene constante, pero el tiempo total aumenta por comunicación, reducción y efectos de ejecución.

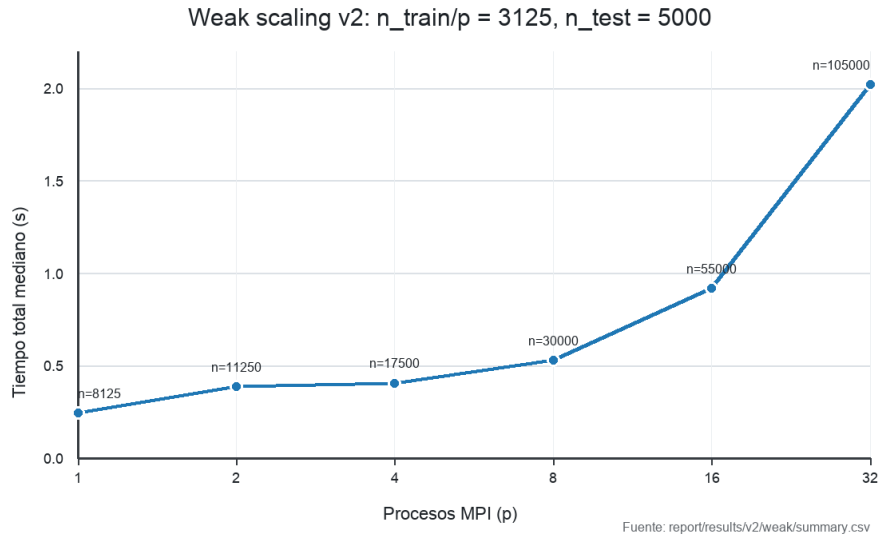


Figura 4: Weak scaling de v2 con $n_{train}/p = 3125$ y $n_{test} = 5000$: el tiempo total mediano aumenta de 0.243 s en $p = 1$ a 2.019 s en $p = 32$, mostrando que la comunicación y el overhead impiden escalabilidad débil ideal.

Idealmente, si solo importara el cómputo local, el tiempo permanecería casi constante al mantener n_{train}/p fijo. En la práctica, el tiempo sube de 0.243 s a 2.019 s entre $p = 1$ y $p = 32$. Esto es coherente con el modelo $O(1 + \log p)$ para las fases globales, aunque la subida real también refleja overhead de Python/MPI y variabilidad del entorno de ejecución.

3.6 FLOPs por segundo

Los FLOPs/s se estiman sobre la región dominante del algoritmo: el cálculo de distancias euclidianas. Para v2 con $n = 105000$, el rendimiento crece desde 8.70 GFLOPs/s en $p = 1$ hasta 50.53 GFLOPs/s en $p = 16$, y luego baja levemente a 49.49 GFLOPs/s en $p = 32$.

p	T_{comp} (s)	Speedup	GFLOPs/s
1	11.086	1.00	8.70
2	5.813	1.91	16.60
4	3.001	3.67	32.16
8	2.044	5.33	47.21
16	1.910	5.72	50.53
32	1.950	5.50	49.49

Cuadro 8: Rendimiento estimado de v2 para $n = 105000$. Los GFLOPs/s aumentan hasta $p = 16$ y se saturan cuando el speedup deja de crecer.

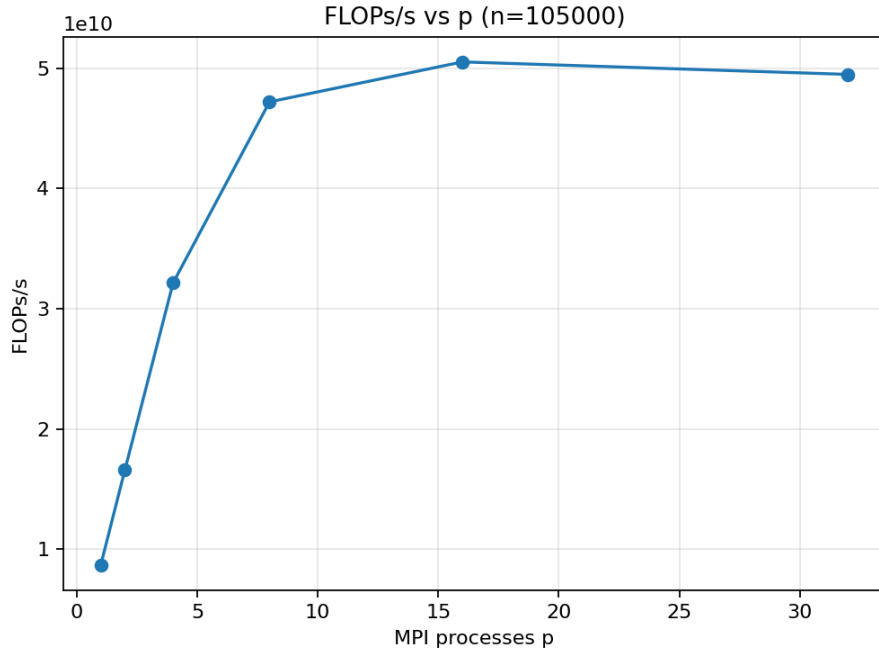


Figura 5: GFLOPs/s de v2 para $n = 105000$: el rendimiento escala hasta $p = 16$ y luego se estabiliza, reflejando el mismo límite observado en speedup.

La saturación de GFLOPs/s se relaciona directamente con la saturación del speedup. Al aumentar p , cada proceso realiza menos distancias, pero el costo de coordinar resultados y mover datos empieza a consumir una fracción mayor del tiempo total.

3.7 Cómputo vs comunicación

En v2, para $n = 105000$, el tiempo de comunicación absoluto aumenta de 0.033 s en $p = 1$ a 0.072 s en $p = 32$. Aunque sigue siendo menor que el tiempo de cómputo, su fracción del tiempo total crece de 0.3 % a 3.6 %. La fase de **scatter** es la más visible en $p = 32$ (0.043 s), seguida por la reducción/fase de reunión Top- K (0.015 s) y el broadcast del test (0.012 s).

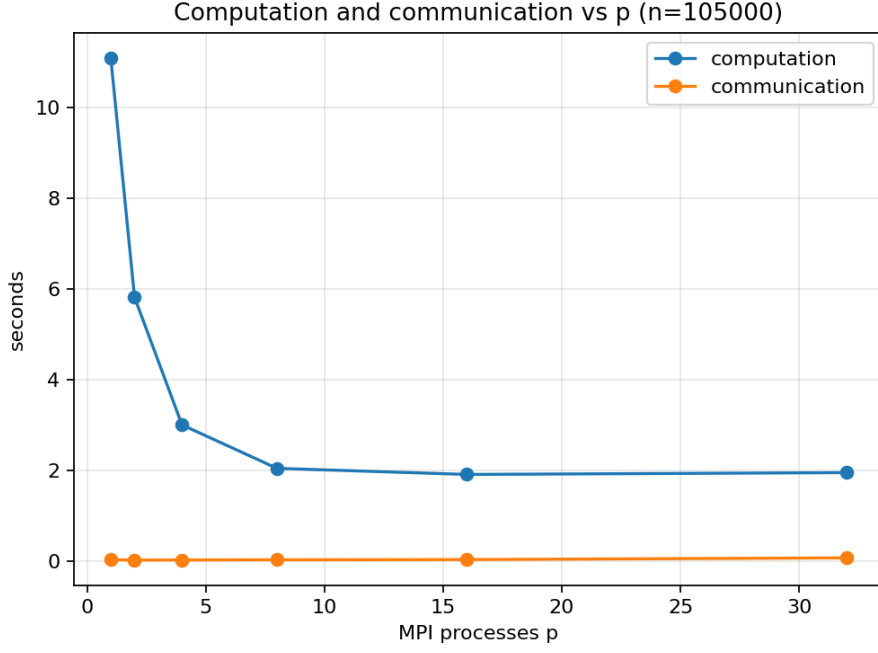


Figura 6: Separación entre cómputo y comunicación en v2 para $n = 105000$: la comunicación crece en proporción relativa al aumentar p y contribuye a limitar la escalabilidad.

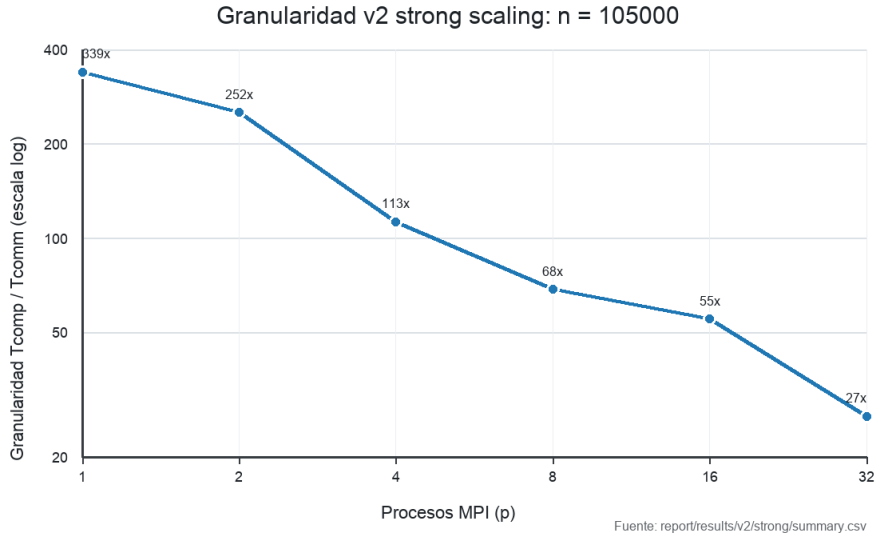


Figura 7: Granularidad de v2 para $n = 105000$, medida como T_{comp}/T_{comm} . La caída de la granularidad al aumentar p muestra que cada proceso realiza menos cómputo útil por unidad de comunicación, lo que contribuye a la saturación del speedup.

La Figura 7 resume este balance mediante la granularidad $G(p) = T_{comp}(p)/T_{comm}(p)$. Una granularidad alta implica que cada proceso ejecuta mucho cómputo por unidad de comunicación. Al aumentar p , cada proceso recibe menos entrenamiento y T_{comp} disminuye; en cambio, T_{comm} no baja de la misma forma porque incluye distribución del entrenamiento, difusión del test y reducción de Top- K . Por ello, la granularidad cae de aproximadamente 339 en $p = 1$ a 27 en $p = 32$, explicando por qué el beneficio marginal de aumentar procesos se reduce y el speedup se satura entre $p = 16$ y $p = 32$.

3.8 Validación experimental de la isoeficiencia

La teoría se contrasta con las mediciones en Khipu para dos versiones: **V2** (reducción con **gather** plano al root) y **V3** (reducción binaria en árbol).

Versión	n_{train}	Mejor p	T_{min} (s)	$S(T_{min})$	S_{32}	E_{32}
V2	10000	16	0.117	8.58	7.24	0.23
V2	25000	32	0.442	6.42	6.42	0.20
V2	50000	8	0.893	6.17	5.86	0.18
V2	100000	16	1.944	5.72	5.50	0.17
V3	10000	32	0.162	25.15	25.15	0.79
V3	25000	32	0.487	23.03	23.03	0.72
V3	50000	32	0.999	23.74	23.74	0.74
V3	100000	32	2.461	22.32	22.32	0.70

Cuadro 9: Resumen de escalabilidad fuerte. El baseline de speedup es $p=1$ de la misma versión y el mismo tamaño.

En la escalabilidad fuerte (Tabla 9), la eficiencia a $p = 32$ es $E_{32} \approx 0,70\text{--}0,79$ en V3 frente a $E_{32} \approx 0,17\text{--}0,23$ en V2. La eficiencia casi constante de V3 al crecer n es consistente con la isoeficiencia alcanzable $n \propto p \log p$ (reducción en árbol, término $O(\log p)$). El **gather** plano de V2 introduce un cuello de botella $O(p)$ en el root, degradando la eficiencia justo como predice el modelo.

Esta lectura no contradice la selección de v2 para el análisis principal. V3 puede mostrar mejor eficiencia relativa e isoeficiencia por su estructura de reducción, pero sus tiempos base son más altos en estos experimentos; por tanto, v2 sigue siendo la opción práctica cuando el criterio es menor tiempo absoluto, mientras que V3 es útil para discutir el comportamiento asintótico de la comunicación.

p	V2 n_{train}	V2 T (s)	V2 GF/s	V3 n_{train}	V3 T (s)	V3 GF/s
1	3125	0.243	12.4	3125	1.067	2.8
2	6250	0.387	15.8	6250	1.183	5.1
4	12500	0.406	30.2	12500	1.263	9.6
8	25000	0.529	46.8	25000	1.326	18.3
16	50000	0.917	53.9	50000	1.456	33.6
32	100000	2.019	49.9	100000	2.292	43.1

Cuadro 10: Escalabilidad débil con 3125 puntos de entrenamiento por proceso, $n_{test} = 5000$, $k = 3$.

En la escalabilidad débil (Tabla 10), con n_{tr}/p constante y n_{te} fijo, el tiempo total crece con p (V2: 0,24s $\rightarrow$ 2,0s; V3: 1,07s $\rightarrow$ 2,29s de $p=1$ a $p=32$). Este crecimiento es coherente con los términos $\log p$ de comunicación: bajo esta forma de weak scaling no se alcanza el caso ideal (tiempo constante) sino $T_p = O(1 + \log p)$, tal como anticipa el análisis, y V3 crece más lentamente que V2 confirmando que la reducción en árbol amortigua mejor el overhead.

3.9 Normalización de la complejidad teórica

La notación Big- O describe crecimiento asintótico, pero no da tiempos en segundos: la expresión de T_p es adimensional y no puede superponerse directamente sobre las mediciones experimentales. *Normalizar* significa introducir las constantes de proporcionalidad que faltan para convertir esa

expresión en segundos, calibrándolas con los propios datos medidos. Con ello puede graficarse $T_{\text{theory}}(p)$ junto a $T_{\text{exp}}(p)$ y verificar que ambas siguen la misma forma.

3.9.1 Modelo con constantes físicas

Se reescribe el tiempo total introduciendo tres constantes: C (tiempo por unidad de trabajo de cómputo), α (latencia por mensaje) y β (tiempo por byte transmitido):

$$T(p) = C \left[\frac{n_{tr}n_{te}}{p} (d + \log_2 K) + n_{te}K \log_2 p \right] + p\alpha + 4\beta n_{tr}d + (\alpha + 4\beta n_{te}d) \log_2 p + (\alpha + 8\beta n_{te}K) \log_2 p.$$

3.9.2 Instanciación con los parámetros del experimento

Para el caso de escalabilidad fuerte con $n_{tr} = 100000$, $n_{te} = 5000$, $d = 64$ y $K = 3$ (dataset `digits`, imágenes 8×8), se evalúan los coeficientes numéricos (con $d + \log_2 K = 65,585$):

$$\begin{aligned} n_{tr}n_{te}(d + \log_2 K) &= 3,279 \times 10^{10}, & n_{te}K &= 1,5 \times 10^4, \\ 4n_{tr}d &= 2,56 \times 10^7, & 4n_{te}d &= 1,28 \times 10^6, & 8n_{te}K &= 1,2 \times 10^5. \end{aligned}$$

Sustituyendo y agrupando los dos términos de comunicación en $\log_2 p$:

$$T(p) = C \left(\frac{3,279 \times 10^{10}}{p} + 1,5 \times 10^4 \log_2 p \right) + 2,56 \times 10^7 \beta + \alpha p + (2\alpha + 1,40 \times 10^6 \beta) \log_2 p.$$

Las únicas incógnitas son la variable p y las constantes C , α y β .

3.9.3 Forma canónica y recuperación de constantes

La expresión anterior se ajusta a un modelo de cuatro formas funcionales:

$$T(p) = a_0 + \frac{a_1}{p} + a_2 \log_2 p + a_3 p,$$

donde cada coeficiente concentra un mecanismo físico distinto:

$$\begin{aligned} a_0 &= 2,56 \times 10^7 \beta \quad (\text{reparto del entrenamiento}), & a_1 &= 3,279 \times 10^{10} C \quad (\text{cómputo}), \\ a_2 &= 1,5 \times 10^4 C + 2\alpha + 1,40 \times 10^6 \beta \quad (\text{bcast} + \text{reduce}), & a_3 &= \alpha \quad (\text{latencia } p\alpha). \end{aligned}$$

Los coeficientes $a_0, \dots, a_3$ se estiman por mínimos cuadrados a partir de los tiempos medidos en $p = 1, 2, 4, 8, 16, 32$. De ellos se recuperan las constantes físicas:

$$\alpha = a_3, \quad \beta = \frac{a_0}{2,56 \times 10^7}, \quad C = \frac{a_1}{3,279 \times 10^{10}}.$$

3.9.4 Validación

Como se ajustan cuatro coeficientes pero solo hay tres constantes físicas, el coeficiente a_2 queda como *ecuación de validación*: debe cumplirse, dentro del error de ajuste,

$$a_2 \approx 1,5 \times 10^4 C + 2\alpha + 1,40 \times 10^6 \beta.$$

Si la relación se satisface y la curva $T(p)$ ajustada reproduce los puntos experimentales, se concluye que el modelo de complejidad captura correctamente el comportamiento observado. En particular, el término $a_3 p$ explica por qué el tiempo deja de mejorar e incluso aumenta para p grande, mientras que a_1/p domina en la región de escalado casi lineal.

3.9.5 Ajuste para la versión v2

Se aplica el procedimiento a la versión v2 (descomposición por entrenamiento con árboles explícitos) en el caso de escalabilidad fuerte con $n_{tr} = 100000$ y $n_{te} = 5000$. Los tiempos totales medidos son:

p	1	2	4	8	16	32
T_p^{exp} (s)	11.119	5.835	3.028	2.088	1.944	2.022

Cuadro 11: Tiempos totales de v2 usados en el ajuste ($n_{tr} = 100000$, $n_{te} = 5000$, $d = 64$, $K = 3$).

Como el término constante de reparto del entrenamiento (a_0) resulta indistinguible con solo seis puntos (el ajuste sin restricción lo lleva a un valor negativo, no físico), se emplea el modelo de tres formas

$$T(p) = \frac{a_1}{p} + a_2 \log_2 p + a_3 p,$$

cuyas constantes quedan exactamente determinadas. El ajuste por mínimos cuadrados da:

$$a_1 = 11,087, \quad a_2 = 0,1608, \quad a_3 = 0,0289, \quad R^2 = 0,9991.$$

Recuperando las constantes físicas mediante $C = a_1/(3,279 \times 10^{10})$, $\alpha = a_3$ y $\beta = (a_2 - 1,5 \times 10^4 C - 2\alpha)/(1,40 \times 10^6)$:

$$C \approx 3,38 \times 10^{-10} \text{ s/unidad}, \quad \alpha \approx 28,9 \text{ ms}, \quad \beta \approx 7,36 \times 10^{-8} \text{ s/byte}.$$

Las tres son positivas y del orden esperado (son constantes *efectivas* que absorben el *overhead* de `mpi4py` y las barreras de sincronización).

La descomposición del tiempo ajustado muestra el cruce entre cómputo y comunicación: en $p = 1$ el cómputo (a_1/p) es el 99,7% del tiempo, pero su peso cae hasta $\sim 38\%$ en $p = 16$ y $\sim 17\%$ en $p = 32$, punto en que la comunicación ($a_2 \log_2 p + a_3 p$) ya domina. Esto valida la predicción teórica: v2 escala casi linealmente mientras el cómputo manda y se estanca cuando la comunicación lo alcanza.

3.9.6 Número óptimo de procesos teórico

Minimizando el modelo ajustado, $\frac{dT}{dp} = -\frac{a_1}{p^2} + \frac{a_2}{p \ln 2} + a_3 = 0$, se obtiene la ecuación cuadrática $a_3 p^2 + \frac{a_2}{\ln 2} p - a_1 = 0$, cuya raíz positiva es:

$$p^* \approx 15,99 \approx 16.$$

Este óptimo teórico coincide exactamente con el óptimo empírico $\arg \min_p T_p^{exp} = 16$ ($T_{16} = 1,944$ s), lo que confirma que el modelo normalizado no solo reproduce la forma de la curva sino también el punto de operación óptimo. Con el criterio alternativo de eficiencia ($E_p \geq 0,5$) el óptimo sería menor, pues para v2 la eficiencia cae por debajo de 0,5 ya en $p = 16$.

3.10 Número óptimo de procesos empírico

La Tabla 12 resume el mejor p experimental para cada tamaño de strong scaling. El mínimo absoluto no siempre coincide con el mejor balance entre tiempo y eficiencia. Por ejemplo, para $n = 105000$, $p = 16$ da el menor tiempo (1.944 s), pero $p = 8$ mantiene una eficiencia más alta (0.67) con un tiempo cercano (2.088 s).

n	Mejor p por tiempo	T_{min} (s)	Speedup	Eficiencia	p recomendado
15000	16	0.117	8.58	0.54	8–16
30000	32	0.442	6.42	0.20	8–16
55000	8	0.893	6.17	0.77	8
105000	16	1.944	5.72	0.36	8–16

Cuadro 12: Número de procesos óptimo para v2 según tiempo total y balance con eficiencia. El mejor tiempo absoluto puede usar más procesos que el punto de mejor aprovechamiento.

Para estos tamaños, $p = 8$ ofrece el balance más defendible: reduce sustancialmente el tiempo frente a $p = 1$ y conserva eficiencias razonables en los tamaños medianos y grandes. Si el objetivo es minimizar tiempo absoluto, $p = 16$ es preferible para $n = 15000$ y $n = 105000$, y $p = 32$ solo resulta atractivo para $n = 30000$ pese a su baja eficiencia.

3.11 Limitaciones y mejoras

Las principales limitaciones son las siguientes. Primero, los experimentos de accuracy tienen una sola ejecución, por lo que validan equivalencia funcional pero no variabilidad estadística. Segundo, la implementación en Python/`mpi4py` introduce overhead adicional frente a una versión C++/MPI. Tercero, los resultados pueden estar afectados por variabilidad del cluster. Cuarto, la comunicación global, la reducción Top- K y la replicación completa del conjunto de test limitan la escalabilidad cuando p aumenta. Finalmente, el rango evaluado llega hasta 32 procesos, por lo que no permite analizar regímenes más grandes.

Como mejoras, se recomienda aumentar el número de repeticiones, reportar desviación estándar o intervalos de confianza en las tablas principales, probar más tamaños de dataset, optimizar el *batching* de test, evaluar una implementación C++/MPI o kernels NumPy más optimizados, y explorar topologías o colectivas MPI alternativas para reducir el costo de difusión y reducción.

4 Conclusiones

Los resultados confirman que el diseño paralelo conserva la correctitud del KNN: para los tamaños de accuracy evaluados, la precisión no cambia con p y las predicciones guardadas para $p = 1, 8, 32$ son idénticas dentro de cada versión. En rendimiento, v2 es la implementación más conveniente para estos experimentos porque obtiene menor tiempo total que v3 en todos los casos de strong scaling.

El strong scaling de v2 muestra una mejora clara hasta $p = 8$ y $p = 16$, pero se satura al llegar a $p = 32$. Para $n = 105000$, el tiempo baja de 11.119 s a 1.944 s en $p = 16$, con speedup 5.72; en $p = 32$ el tiempo sube a 2.022 s y la eficiencia cae a 0.17. Esto coincide con el análisis teórico: el cómputo local disminuye al repartir el entrenamiento, pero la comunicación, la reducción Top- K y el overhead de ejecución limitan el beneficio marginal de usar más procesos.

El análisis de granularidad refuerza esta conclusión: al aumentar p , la razón T_{comp}/T_{comm} disminuye, lo que confirma que el cómputo local se reduce más rápido que la comunicación y explica la pérdida de eficiencia para valores grandes de p .

En weak scaling, mantener $n_{train}/p = 3125$ no mantiene el tiempo constante: v2 pasa de 0.243 s en $p = 1$ a 2.019 s en $p = 32$. Por tanto, el algoritmo escala de forma útil para tamaños moderados, pero no ideal. En estos datos, $p = 8$ ofrece el mejor balance general entre tiempo y eficiencia, mientras que $p = 16$ puede ser preferible cuando se prioriza el menor tiempo absoluto en los tamaños más grandes.

Referencias

- [1] Lisandro Dalcin. *mpi4py: MPI for Python*. Documentación oficial. Disponible en: <https://mpi4py.readthedocs.io/>.
- [2] Scikit-learn developers. *The digits dataset*. Documentación oficial. Disponible en: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html.
- [3] Mukanjy0. *par-knn*. Repositorio del proyecto. Disponible en: <https://github.com/mukanjy0/par-knn>.
- [4] Anthropic. *Claude*. Asistente de inteligencia artificial usado como apoyo para implementación, depuración y revisión del reporte. Disponible en: <https://claude.ai/>. Consultado en 2026.
- [5] OpenAI. *Codex*. Asistente de programación usado como apoyo para implementación MPI, automatización de experimentos y edición del documento. Disponible en: <https://openai.com/codex/>. Consultado en 2026.